

Google Reverse Video Search

Peter Trinh

ptrinh32@gatech.edu

Abstract— This paper explores the concept of the inclusion of reverse video searching as a possible new addition to Google’s current search engine capabilities along with Google’s reverse image search. The reverse video searching algorithm utilizes Google’s current reverse image search to analyze a large amount of images taken from breaking down the frames of videos so that a mass search of a video is completed to identify the source of the video.

1 PROBLEM SPACE

1.1 Problem and Context

Society nowadays is completely immersed in social media and entertainment. Large social platforms designed for media consumption such as Youtube, Instagram, and Tiktok are constantly growing in both consumers and creators. Tiktok alone boasts over 8.6 billion videos uploaded in 2021, which has only grown since then (Ch, 2024). Video marketing has had a massive increase in importance as 67% of marketers are relying more on videos (Valencia, 2024). With such growth in the inclusion of video in society’s entertainment and business industries, the ability to be able to identify the source of a video has become a crucial skill that is still severely lacking. For consumers who frequently use platforms such as Instagram and Tiktok where there are multitudes of videos being reposted from the original source, identifying where a clip originated and finding similar videos are an important part of the entertainment of the social platform. There have been many occasions in which I personally have not been able to find the origin of a clip due to so many reposts and copied videos of the original. This issue makes it difficult for fans to be able to support his or her original content creators and videos. Another issue with the inability to easily locate a video’s original source involves businesses and copyright infringement. Copyright issues are an everyday occurrence that video creators must contend with. With reliable video reverse search, individuals would be able to find illegitimate uses of his or her content or check and ensure that he or she is not violating any copyright issues themselves. Reverse video searching should be a viable implementation that is completely possible with current technology that should be able to help resolve these issues and improve the video creation and entertainment industries.

1.2 Proposed Task

My proposal is the extension of Google's current reverse image search that is available on Google's search engine. Currently, Google's reverse image search is able to take images and do a search on posts that are relevant or similar to the given image. However, Google does not have any functionality to use the search engine with an entire video. If an individual desires to use Google to search for a specific video, he or she would need to take screenshots of the video and use the image to reverse image search. However, since a 10 minute video can have upwards of 10,000 frames, doing a Google search on a singular frame would not be able to give the most accurate search engine results. To be able to accurately get search engine results of an entire video, the frames of the entire video would need to be used within a search engine to find all instances of every possible frame. Despite this, doing 10,000 Google reverse image searches for a singular 10 minute video would be unnecessarily inefficient. Strategically utilizing a small amount of the specified frames would be able to yield an overall average of the entire video would be able to give accurate results within a reasonable time. My proposal is to implement this system into the existing Google search engine as an optional extra alternative to Google's regular search engine and Google's reverse image searching.

2 INITIAL NEED FINDING ANALYSIS AND PRELIMINARY EVALUATION

2.1 Need Finding Exercise

For the needfinding exercise, a usage survey was completed to be able to gather data in which the problem space and context will be analyzed to further determine the issue and possible solutions. The survey was conducted with the assistance of five individuals who are either consumers of social media and entertainment or works within the video industry. The design of the survey was made to give participants the most flexibility in describing their opinions with their challenges when working with videos and Google searches involving the videos. The survey in which the participants were given can be seen below and the participants' respective answers are available within the appendix section of this paper.

2.2 Usage Survey

Survey interview questions:

- What is your gender?
 - Male
 - Female
- What is your age range?
 - 0 - 18

- 18 - 35
- 35 - 45
- 45 - 60
- 60+
- Are you an avid user of social media video platforms, such as Instagram or Tiktok?
 - Yes
 - No
- Do you work within the video creation industry?
 - Yes
 - No
- Have you ever had to identify the source of a video without any information on the creator or name of the video?
 - Yes
 - No
- Have you ever used Google's reverse image search to find said video in the previous context?
 - Yes
 - No
- Please give your opinion on being able to reverse video search the entire video of said video in the previous context (Short answer question).
 - _____

2.3 Heuristic Based Evaluation

According to the survey results of the needfinding exercise, the heuristic based evaluation can be defined into pros and cons of the current proposed task. The positives of the proposed tasks revolves around the main issue of being able to find the original source of a video. The functionality of this alone can alleviate a wide variety of issues, such as copyright issues and finding related content. Another pro that is supported by the survey results is the improvement over the current method of finding the original source of a video. This current method is to take screenshots of a video and individually place them within Google's reverse search image to find related content one at a time. The proposed task is supported due to its increased speed in completing the task. On the other hand, one of the possible concerns to the proposed tasks include having to download an entire video to be able to complete the task, which would be heavily demanding when working with devices that have low data storage.

2.4 Preliminary Empirical Measurements

The preliminary empirical measurements are gathered through both the usage survey results and publicly available data. Based on the survey results, every participant of the survey has had to identify a video without having any source related information whatsoever. The only possible way to be able to currently find the source of a video is through reverse video searching using a singular image taken from the image and searching through Google, Berify, Yandex, TinEye, and Shutterstock (Seo Manager, 2021). This shows that there is almost no set empirical data that can be found from the proposed task of reverse video searching using an entire video itself. The best comparison would be utilizing the time from an individual to manually reverse image search frames from a video compared to the proposed task.

3 BASELINE DESIGN METHOD AND MEASUREMENT RESULTS

3.1 Prototype Pseudo code

- The complete code can be seen in the appendix

Main Function:

1. Initialize data storage (empty list).
2. Extract 100 evenly spaced frames from video:
 - a. For each frame:
 - i. Extract the frame at a specific interval.
 - ii. Save the frame as an image in the output folder.
3. For each extracted image:
 - a. Perform Google image search based on the image name (or a related query).
 - b. Collect search result data (e.g., title, link, title length).
4. Save collected search results data to be displayed to the user.

Frame Extraction (extract_frames):

1. Open video file.
2. Calculate the number of frames and determine frame intervals based on total frames and number of images (100).
3. Loop through the video:
 - a. At each interval, extract a frame.
 - b. Save the frame as a .jpg file in the output folder.
4. Repeat until 100 frames have been extracted.

Google Image Search (google_image_search):

1. Connect to Google's Custom Search API.

2. For each query (image name):
 - a. Retrieve the top 10 search results.
 - b. Extract relevant features (e.g., title, link, title length).
 - c. Store results in a list.
3. Return the collected results.

3.2 Implementation

The baseline design focuses solely on the main task of incorporating in a video to Google's Reverse Image search to be able to find a video's original source. The implementation first requires the desired video to be completely downloaded and available to be accessed by the program. The entire length of the video will then be split into 100 individual sections. Within each section, the first frame of the section will be extracted to be used in the Google Reverse Image search. Google's api allows this to be done completely within the program. Each image search will display an unknown number of search results; however, only the first 10 results will be extracted. In total there should be 1000 search results in total after all 100 frames are reverse image search. These search results will be compiled together and displayed to the user.

3.3 Preliminary Experimental Evaluation

	Google Reverse Image Search (10 images)	Berify Reverse Image Search (10 images)	Proposed Application Reverse Video Search (100 images)
10 sec video	3 mins 9 sec	3 mins 17 sec	< 30 sec
1 min video	3 mins 43 sec	3 mins 24 sec	1 min 3 sec
5 min video	3 mins 22 sec	3 mins 43 sec	2 min 37 sec
30 min video	3 mins 13 sec	3 mins 6 sec	4 min 14 sec
1 hr video	3 mins 35 sec	3 mins 26 sec	10 min 55 sec

Figure 1—Time taken from timed test of each method. Google Reverse Image Search and Berify Reverse Image Search took 10 images, while the Proposed Application took 100 images.

This table is created using the data that was taken through the comparison between Google Reverse Image Search, Berify, and the proposed application of reverse image searching. The context of this table is set so that 5 videos of 10 seconds, 1 minute, 5 minute, 30 minutes, and 1 hour were used for comparison between the five methods. The Google Reverse Image Search and the Berify Reverse Image Search were then used on each video. 10 images of each video were used for the search and the total time to get all the search results were then compared to the

time of the reverse video search. The reverse video search, on the other hand, would do the reverse search of 100 images from the video and the total time to get all the search results would then be compared. The data clearly shows that the longer the video the slower the reverse video search would take to get all the search results. The reverse image search, however, shows consistent time all throughout despite the video time. The data supports that the application significantly slows down as the length of the video increases. This is directly related to the process of the reverse video search since the baseline design requires the user to completely download the video for the application. However, the reverse image searches only require the user to take images throughout the video. This can be a possible improvement to be refined in the design refinement stage.

3.4 Heuristic-Based Evaluation

Analyzing the heuristic based evaluation from the survey results along with the baseline implementation, there are key challenges that need to be overcome to improve the baseline design in the design refinement stage. Efficiency, technical feasibility, functionality, and user experience are the main design principles that are in need of improvement. The current efficiency in the baseline design is severely lacking since for the application to be used, videos need to be downloaded completely to be broken down into frames for analysis. This will result in larger video files to always be inefficient compared to smaller videos and will take much longer for the program to complete. Technical feasibility is restricted through the specific set up of the baseline design. This design requires the video that's going to be reverse searched to be downloaded and placed inside the program, which does not have the flexibility that a proper application should have. This is another improvement that can be explored in the refined design. Functionality of the baseline design can be observed to be properly working after analyzing the baseline design. However, part of the search result display does not factor in any type of display preference of formatting for the user. This can be viewed as a partial failure in functionality since although the search results were all returned properly, the user themselves will have to manually sort and identify the relevant and irrelevant search results. Finally, the heuristic evaluation clearly demonstrates that user experience was not a priority for the baseline design. The baseline design focused more heavily on functionality of the proposed task instead of user experience. The baseline design lacks an user interface and is mainly run completely through the use of the command line, so the refined design or alternative design can be improved over the baseline design by adding in this factor.

3.5 Design Refinement Brainstorm and Alternative Strategies

Although the baseline design is complete and functionally appropriate for the proposed task, there are still many possible enhancements and refinements that can be done to improve the

overall design. Currently the design simply outputs every single possible google image search for the user, which results in one thousand results since the design returns 10 search results for each frame captured. To better restrict this, the search results can be sorted and displayed through relevance to the image and the frequency in its occurrence. In addition, the current baseline design requires the video to be downloaded and input into the program. Other improvements would be to enable the use of a link instead of having to download the entire video itself.

4 REDESIGN THROUGH REFINEMENTS

4.1 Design Refinement

Following the design refinements from the brainstorming, the original baseline design was modified to be able to account for the display of the google search results and the video link. The program will now be able to take in not just a downloaded video but also a video link as well. In addition, the search results from google will now be displayed based on relevance and frequency of the search result to the video itself.

4.2 Pseudo code

- The complete code can be found in the appendix

Main Function:

1. Initialize data storage (empty list).
2. Extract 100 evenly spaced frames from video:
 - a. For each frame:
 - i. Extract the frame at a specific interval.
 - ii. Save the frame as an image in the output folder.
3. For each extracted image:
 - a. Perform Google image search based on the image name (or a related query).
 - b. Collect search result data (e.g., title, link, title length).
4. Save collected search results data to be displayed to the user.

Frame Extraction (extract_frames):

1. Open video file.
2. Calculate the number of frames and determine frame intervals based on total frames and number of images (100).
3. Loop through the video:
 - a. At each interval, extract a frame.
 - b. Save the frame as a .jpg file in the output folder.
4. Repeat until 100 frames have been extracted.

Google Image Search (google_image_search):

1. Connect to Google's Custom Search API.
2. For each query (image name):
 - a. Retrieve the top 10 search results.
 - b. Extract relevant features (e.g., title, link, title length).
 - c. Store results in a list.
3. Return the collected results.

Compile and Sort Results

1. Collect all result links across all images.
2. Count the frequency of each unique link.
3. Sort the links by frequency in descending order.

Prepare Sorted Results

1. For each unique link:
 - a. Find a representative result (title, context).
 - b. Append the result with metadata and frequency count to the sorted list.

Display and Save results

1. Print the sorted results to the console.
2. Save the results to a text file with metadata and frequency counts.

4.3 Implementation

The refined design improves upon the baseline design to create an application designed to process videos by extracting frames from the taken video and reverse image search the images through Google's reverse image search api. The baseline design only implemented the core logic of the application where the videos were broken down to be analyzed with the reverse image search. However, the refined design is divided into two main components: a backend implemented in Python with Flask and a frontend developed using HTML, CSS, and JavaScript. These components create a seamless workflow that allows users to experience a proper user interface to analyze videos and retrieve related visual content efficiently, unlike the baseline design.

The backend starts by setting up a Flask endpoint that handles HTTP POST requests. This backend was implemented in the refined design to connect the refined design's frontend to the

actual video analysis code. Users can provide a video via a URL or upload a local video file. If a URL is submitted, the backend downloads the video using Python's requests library, employing streaming to manage large files efficiently. For uploaded files, the video is saved directly to a temporary directory so that large video files will not need to be downloaded like the baseline design. This issue was one of the core problems that arose during the baseline design analysis and brainstorming of the refined design. Once the video is ready, OpenCV is used to extract evenly spaced frames by calculating the interval required to divide the video into the desired number of frames. The current number of frames taken is 100 frames; however, this number can be adjusted based on the performance of the refined design. Each extracted frame is saved as a .jpg image in a designated folder. For each extracted frame, the program uses Google's Custom Search API to perform an image search. The results for each frame include the image title, link, and context URL. These results are collected into a dictionary, where each frame's filename maps to its respective search results. To rank the results based on relevance, all links are flattened into a single list, and Python's Counter is used to count the frequency of each unique link across all frames. The results are then sorted by frequency, prioritizing links that appear most often. This frequency-based sorting ensures that the most relevant results are highlighted for the user. The backend sends these organized results as a JSON response to the frontend.

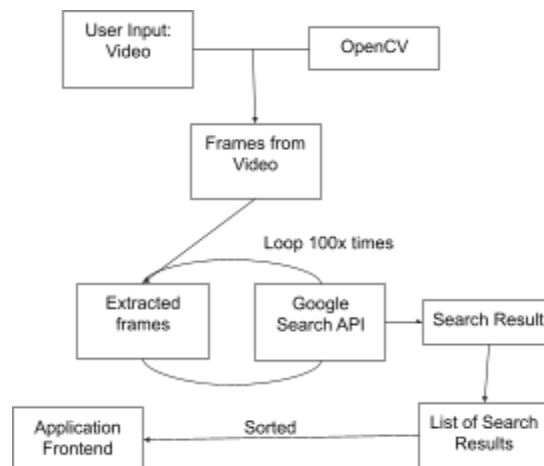


Figure 2—Flowchart Implementation of Refined Design

The frontend interface provides users with a way to interact with the system. It includes a form where users can either input a video URL or upload a file directly. JavaScript manages the form submission and sends the data to the backend via asynchronous fetch requests, ensuring smooth operation without page reloads. While the video is being processed, a status message updates the user on the progress. Once the results are returned, the webpage displays them in a structured layout so the user is able to clearly understand the results. Each result includes the image title, a clickable link to the source, and the frequency count.

4.4 Proposed Design Refinement Possible Failures

The design refinements have improved upon a lot of the problems and failures of the baseline design. However, there are still possible failures that have not been accounted for within the refined design. This new design incorporates a frontend for the application so that users will not have to use the command line for the project. Despite this, the frontend of the application is only roughly done for the application, which results in possible error in design such as text overflow. In addition, this design does not account for any errors throughout the entire application. Errors should arise with the Google image search or the upload of a file that is in the wrong file type. These are some basic errors that the user may come across that the refined design did not account for. The last situation that may cause failures for the refined design revolves around the core of the application itself. Just like any Google search, there is a possibility for the reverse video search to ultimately return no relevant or correct search results. This situation stems from images taken from the video not relating to any google searches. This will give the user no search results. However, since a search engine not being able to give any search results is a common occurrence in any search engine, the context of this failure cannot be attributed to the failure of functionality of the refined design.

5 EVALUATION PLAN

5.1 Qualitative and Empirical Evaluation

For the evaluation of the refined design, a qualitative and empirical evaluation will be utilized to analyze the application. A qualitative evaluation will be done to assess this program due to its effectiveness in giving insight to the user's perspective and providing a detailed understanding of how well the application is able to complete its given prompt task from the user. An empirical evaluation will be used alongside a qualitative evaluation because empirical evaluations are able to offer the most concrete evidence in the application's performance and effectiveness.

While both evaluations have its advantages and disadvantages, by utilizing both evaluations, the data collected from the evaluations will be able to best provide an understanding of the design's current successful features and areas in need of improvement. In addition, the qualitative evaluation will be able to identify problems that the empirical evaluation is not able to realize. For instance, the program heavily relies on Google Custom Search API for retrieving image-related information. While empirical metrics might measure API response times or the number of results retrieved, they cannot assess whether the results are truly helpful or aligned with user expectations. Similarly, metrics such as processing speed can highlight specific strengths and areas for improvement, enabling the developers to make data-driven decisions that qualitative evaluations do not show. Users might have subjective opinions about the relevance of

search results or how effectively the program organizes and presents them and empirical data provides measurable objective data to offer insight to developers of the application. This is why using qualitative and empirical evaluations will offer the most feedback on improving the application.

5.2 Evaluation Metrics

The evaluation metrics of the qualitative evaluation are designed to gather user feedback on the application's usability, functionality, performance, and satisfaction. A survey will be used to gather data from participants about the refined design. The usability of the application is assessed on how intuitive and easy the application is to use for the user. The navigation, user input, and direction of the application should all be taken into account. The functionality questions evaluate the actual tasks that the user has and whether or not the user is able to complete the task. The main objective of the application is to be able to process videos accurately and generate relevant sorted Google search results from images taken from the videos. The performance of the application only has a singular question that focuses on the application's overall speed and how quickly the user is able to complete his or her task. The last metric for evaluation in the qualitative survey is the user's overall satisfaction and user experience with the application itself. These questions are used to identify areas of success and opportunities for enhancement based on user opinions and suggestions. Together, these questions allow for a comprehensive evaluation, helping to pinpoint specific strengths and weaknesses across all critical aspects of the application. The evaluation metric of the empirical evaluation is much simpler than the qualitative evaluation since it focuses heavily on only performance. The performance of the refined design will be determined through the comparison of the baseline design's preemptive empirical evaluation to determine whether or not the refined design was able to reduce the time needed for completing the prompted task.

5.3 Empirical Evaluation Test

The empirical evaluation will be done in comparison with the baseline design's preemptive empirical evaluation. The context of the evaluation will be set up to mimic the process of the previous evaluation. The application will be run with five videos at varying overall times. The times of each video will be 10 seconds, 1 minute, 5 minutes, 30 minutes, and 1 hour respectively. The total time for the entire process of analyzing the video and returning the search results will be compared to the baseline design's time along with Google's reverse image search and Berify's reverse image search. This evaluation test will allow the current process for reverse image search, the baseline design's reverse video search, and the refined design's reverse video search to be compared based on time.

5.2 Qualitative Survey Context

Due to the limitations of the application, such as setting up the search engine id and api key, the evaluation survey of the refined design is restricted to individuals who are able to use the application in person. The survey will be done after the individuals have an adequate amount of time to familiarize themselves with the refined design and have completely finished using the entire application. Each participant will be asked to have at least one specified video of unknown origin available with them to be used with the application for the survey. Once the survey participants have finished their prompted tasks, he or she will then complete the following survey shown below.

5.3 Qualitative Survey

Survey interview questions:

These questions focused on usability

- How easy was it to navigate and use the application?
 - Scale (1 Very Hard - 5 Very Easy)
- Did you encounter any issues while submitting a video URL or uploading a video file (If yes, please describe)?
 - Yes
 - Please describe _____
 - No
- Was the results section easy to read and understand?
 - Yes
 - No

These questions focused on functionality

- Did the application successfully process the video you provided?
 - Yes
 - No
- Were the Google search results for each frame relevant and useful?
 - Yes
 - No
- Did the sorting of results based on frequency meet your expectations for relevance?
 - Scale (1 Very Not Relevant - 5 Very Relevant)

This question focused on performance

- How would you rate the speed of the application in processing the video and displaying results?
 - Scale (1 Very Not Fast - 5 Very Fast)

These questions focused on satisfaction

- How satisfied are you with the overall experience of using this application?
 - Scale (1 Very Not Satisfied - 5 Very Satisfied)
- What areas do you think need improvement?
 - _____

5.5 Biases

For this evaluation survey, selection bias and confirmation bias are the two most likely biases to come up. Selection bias is a bias in which the selection of the participants of a survey is not able to properly act as a sample of the population. Without proper randomization of the participants, neutrality of the participants cannot be achieved. The participants in this selection may not be the best to represent the application's intended user base since the participants are required to be able to run the application in person. Such a restriction limits the number of people the participants can be. To minimize selection bias, the survey will be distributed to a diverse group of participants that mirrors the application's target audience as best as possible while still satisfying the application's current usage conditions. In addition to selection bias, confirmation bias can also be likely to be found in this survey. Confirmation bias can occur if the survey questions are phrased in a way that leads participants to provide responses that confirm the developer's expectations or desired outcomes. To best mitigate such a bias, the wording and design of the questions for the survey will need to be completely neutral and encourage honest feedback of the application. Additionally, including open-ended questions allows participants to share thoughts freely, without being hindered by predetermined notions and opinions from the survey and application developer. These strategies will be implemented to help ensure that the survey results are best able to represent the population's opinion on the application.

6 EVALUATION EXECUTION AND RESULTS

6.1 Qualitative Survey Results

Looking at the survey data, multiple conclusions can be drawn from the participants' opinions of the application. Across the multiple evaluation metrics, it can be determined that there is an overall positive reaction to the application.

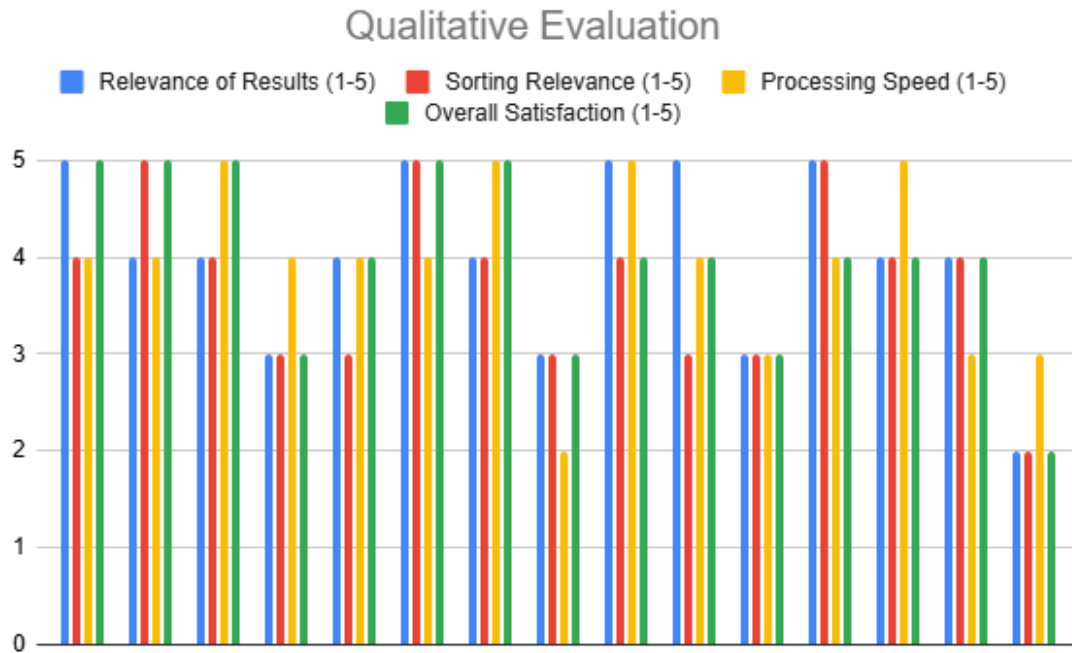


Figure 3—Bar Graph of Evaluation Metrics from Survey Results

Usability scored well overall from the graph averaging 4 out of 5, with most users finding the application easy to navigate. However, a few participants noted issues such as difficulties in uploading video files, suggesting the need for a more intuitive interface and better file handling.

Functionality received generally high marks, as most users successfully processed their videos and found the results section easy to interpret. However, there were definitely some search results that had lower relevance to the videos processed. This led to a lower score in functionality with an average sorting relevance score of 3.8, indicating a need for refining the algorithm or data presentation.

The performance metric was the metric that had the lowest ratings compared to the other metrics with an average of 3.5 out of 5. Smaller videos were able to be processed quickly and efficiently. However, larger videos were the main obstacle that stemmed from a lower score than the other metrics. This emphasizes the necessity for optimization in handling and analysis in further refinements.

Satisfaction ratings averaged around 4.3, showing overall positive user sentiment, though areas for improvement include enhancing the speed and relevance of results, as well as addressing technical issues like failed uploads. The survey also gathered data on suggestions that can be incorporated into further design iterations such as faster upload speeds, clickable result links, and improved sorting mechanisms. The results of the survey clearly indicate that while the

application is functional and user-friendly, targeted improvements could significantly elevate its efficiency and functionality, particularly in areas like performance and result relevance.

6.2 Empirical Evaluation Results

	Google Reverse Image Search (10 images)	Berify Reverse Image Search (10 images)	Proposed Application Reverse Video Search (100 images)	Refined Design Application Reverse Video Search (100 images)
10 sec video	3 mins 9 sec	3 mins 17 sec	< 30 sec	< 5 sec
1 min video	3 mins 43 sec	3 mins 24 sec	1 min 3 sec	22 sec
5 min video	3 mins 22 sec	3 mins 43 sec	2 min 37 sec	1 min 3 sec
30 min video	3 mins 13 sec	3 mins 6 sec	4 min 14 sec	3 min 17 sec
1 hr video	3 mins 35 sec	3 mins 26 sec	10 min 55 sec	8 min 9 sec

Figure 4—Time taken from timed test of each method. Google Reverse Image Search and Berify Reverse Image Search took 10 images, while the Proposed Application Baseline Design and the Refined Design took 100 images.

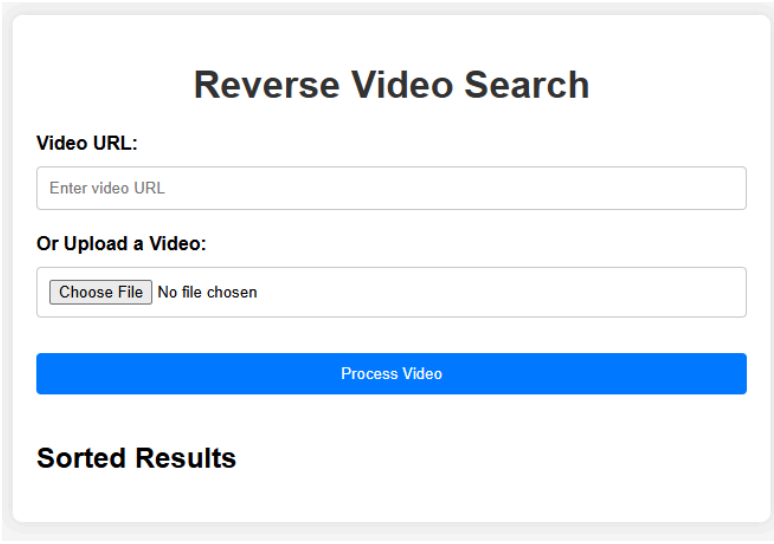
According to the graph above, there is a clear distinction between the times taken from each method. The performance comparison between the baseline design, the refined design, and the other tools (Google Reverse Image Search and Berify Reverse Image Search) reveals significant differences in speed and scalability for video processing tasks. Focusing on only just the baseline design versus the refined design, the refined design outperforms the baseline design in speed, especially with regards to longer videos. For example, it processes a 10-second video in under 5 seconds and a 1-hour video in just over 8 minutes, while the baseline requires approximately 30 seconds and nearly 11 minutes for the same tasks. This suggests that the refined design was able to more efficiently handle the video processing and frame extraction from the videos, since the baseline and refined designs both utilize the Google search API. Looking at the reverse image search times for Google and Berify, although both of these methods had an average time of 3 to 4 minutes regardless of video length, both methods required the user to manually search every single individual frame. This is counterintuitive of the baseline design and refined design which was developed to mainstream the process and remove the user's manual labor. Taking this into

account, the increased time for the two designs are negligible since it completely removed the manual labor. The baseline design, while better suited for video analysis than Google or Berify, still shows room for optimization as its performance scales less efficiently for longer videos. The refined design however, demonstrates a clear advantage by achieving shorter processing times across all video lengths, emphasizing its suitability for applications requiring fast, large-scale video analysis. This makes the refined design a more practical and efficient choice overall.

6.3 Next Round of Design

Based on the results found from the qualitative and empirical evaluations, there are still many improvements possible for the next iteration of the development cycle. The empirical data clearly shows that the processing speed of the videos are still significantly higher than what is to be expected. Longer videos tend to take much longer for the analysis to return search results than shorter videos. Further iterations of the design will need to focus heavily on efficiency and increasing the overall speed of the application to be able to allow users to get the search engine results within a timely manner. In addition, the qualitative survey supports the addition of multiple error checks throughout the program, such as the file upload error and the url search error. This requires more in depth quality assurance of the application so that all bugs are removed. The last few possible changes that could be added to the next development cycle would revolve around user experience enhancement and overall satisfaction of the application, such as making clickable links and more description in the google search results. All of these factors can greatly further enhance the current application design and are all possible additions to the next application design.

6.4 Prototype



Reverse Video Search

Video URL:

Enter video URL

Or Upload a Video:

Choose File No file chosen

Process Video

Sorted Results

Figure 5— Application Frontend Design

This is the user interface for the prototype of the application. Following the refined design, the user is given an option of either entering in the video url or uploading a video directly. The user will then click the process video button and the video will then be analyzed. The results of the video can be seen in Figure 6.

The screenshot displays a web application titled "Reverse Video Search". It features two input methods: a "Video URL:" field with a text input containing a YouTube link, and an "Or Upload a Video:" section with a "Choose File" button and "No file chosen" text. A prominent blue "Process Video" button is located below the input fields. The "Sorted Results" section lists three items, each with a title, a URL, and a frequency value. The results are sorted by frequency, with the highest frequency at the top.

Rank	Title	URL	Frequency
1.	Rick Astley - Never Gonna Give You Up (Official Music Video)	https://www.youtube.com/watch?v=dQw4w9WgXcQ	5
2.	Never Gonna Give You Up	https://en.wikipedia.org/wiki/Never_Gonna_Give_You_Up	3
3.	Rick Astley - Never Gonna Give You Up (Remastered 4K 60fps,AI)	https://www.youtube.com/watch?v=o-YBDTqX_ZU	2

Figure 6— Application Returned Search Results

This figure shows how the search results will be returned to the user based on the video searched. In this example, the video that is searched is Rick Astley’s music video of his song “Never Gonna Give You Up”. The search results returned multiple search results; however, this image currently only showed 3 results. The rest of the results can be seen by scrolling down on the results. The returned results displayed the name or title of the webpage returned, the url link of the google search result and the frequency in which the search result appeared. The higher the frequency the more relevant the search result is to the processed video.

7 CONCLUSION

This project's entire process ranging from design to development and prototyping was extremely beneficial in the understanding of new technologies. Being able to develop from scratch a working prototype of a new possibly marketable idea helped solidify my understanding of the material. In addition, the exposure towards the new technologies that I have never worked with before such as Google's search API allowed me to further advance my studies and learn about different fields within the software development industry that I have not been exposed to. One of the most rewarding aspects of this project was to be able to contribute 100% towards the development of the project which I rarely have experience on. There were many aspects of the software development process that I was unaware of that this project has shown me.

This project utilizes several foundational Internet computing concepts and technologies within its development. The architecture employs a web-based client-server model, where the Flask backend provides RESTful endpoints that the frontend interacts with through asynchronous HTTP requests. Another concept involves the integration with cloud APIs, particularly Google's Custom Search API, to perform reverse image searches for extracted frames. There are many other technological concepts throughout the rest of the project that were used to develop this application so that it can efficiently process videos and return search engine results to the user.

This application has a very high future potential since it's able to be integrated into any known search engine. With support from communities and new features there is a strong possibility of this project's concept to be developed further. The search results can be improved and one promising aspect is the development of advanced search capabilities, such as integrating multiple reverse image search engines, audio recognition, and enabling side-by-side comparisons of results. This would provide users with more comprehensive search options and insights, which will also improve the accuracy of the search results. Enhancing the user interface with filters, better visualization tools like graphs and timelines, and more interactive elements could further improve usability and attract a broader audience. The different contexts in which this project's concepts are applicable are immense and there should be plenty of improvements that can be made on the application.

8 REFERENCES

1. Valencia, A. (2024, March 7). *How To Use Reverse Video Search (& Why It's Useful)*. Search Engine Journal. <https://www.searchenginejournal.com/how-reverse-video-search/464654/>
2. Ch, D. (2024, August 3). *TikTok Statistics: Revenue & Usage (Updated August 2024)*. SendShort - Create Viral Shorts Instantly with SendShort. <https://sendshort.ai/statistics/tiktok/>

3. Seo Manager. (2021, March 25). *Reverse Video Search – 6 Different Methods To Follow*. DigitalGuider. <https://digitalguider.com/blog/reverse-video-search/>

9 APPENDIX

9.1 Heuristic Evaluation Survey Raw Results

[Link Here](#)

9.2 Qualitative Evaluation Survey Raw Results

[Link Here](#)

9.3 Complete Code

```
import os
import math
import cv2
from flask import Flask, request, jsonify
from googleapiclient.discovery import build
from collections import Counter

app = Flask(__name__)

outputFolder = 'extracted_images'
uploadFolder = 'uploads'
apiKey = 'your_google_api_key'
searchEngineID = 'your_cse_id'
imageNum = 100

os.makedirs(outputFolder, exist_ok=True)
os.makedirs(uploadFolder, exist_ok=True)

def extract_frames(video_path, outputFolder, imageNum):
    if not os.path.exists(outputFolder):
        os.makedirs(outputFolder)
    video = cv2.VideoCapture(video_path)
    total_frames = int(video.get(cv2.CAP_PROP_FRAME_COUNT))
    frame_interval = max(1, math.floor(total_frames / imageNum))
    count = 0
    for i in range(0, total_frames, frame_interval):
```

```

        video.set(cv2.CAP_PROP_POS_FRAMES, i)
        success, frame = video.read()
        if not success:
            break
        output_filename = os.path.join(outputFolder, f"frame_{count}.jpg")
        cv2.imwrite(output_filename, frame)
        count += 1
        if count >= imageNum:
            break
    video.release()
    return count

def google_image_search(query):
    service = build("customsearch", "v1", developerKey=apiKey)
    res = service.cse().list(
        q=query,
        cx=searchEngineID,
        searchType='image',
        num=10
    ).execute()
    results = []
    if 'items' in res:
        for item in res['items']:
            title = item['title']
            link = item['link']
            context_link = item['image']['contextLink']

            results.append({
                'title': title,
                'link': link,
                'context_link': context_link,
            })
    return results

def compile_and_sort_results(image_results):
    all_links = []
    for results in image_results.values():

```

```

    for result in results:
        all_links.append(result['link'])
link_counter = Counter(all_links)
sorted_links = link_counter.most_common()
sorted_results = []
for link, count in sorted_links:
    for results in image_results.values():
        for result in results:
            if result['link'] == link:
                sorted_results.append({
                    'title': result['title'],
                    'link': result['link'],
                    'context_link': result['context_link'],
                    'count': count
                })
                break
            else:
                continue
        break
return sorted_results

```

```

@app.route('/process_video', methods=['POST'])
def process_video():
    video_url = request.form.get("video_url")
    video_file = request.files.get("video_file")
    if not video_url and not video_file:
        return jsonify({"error": "No video URL or file provided."}), 400
    try:
        if video_file:
            video_path = os.path.join(uploadFolder, video_file.filename)
            video_file.save(video_path)
        elif video_url:
            video_path = video_url
        extracted_frames = extract_frames(video_path, outputFolder, imageNum)
        image_results = {}
        for image_filename in os.listdir(outputFolder):
            if image_filename.endswith('.jpg'):

```

```

        query = image_filename
        results = google_image_search(query)
        image_results[image_filename] = results
        sorted_results = compile_and_sort_results(image_results)
        return jsonify({"results": sorted_results, "frames_extracted": extracted_frames})
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(debug=True, port=5000)

```

9.3 Frontend Code

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Video Frame Analysis</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
      padding: 0;
      background-color: #f5f5f5;
    }
    .container {
      max-width: 600px;
      margin: 0 auto;
      padding: 20px;
      background: #ffffff;
      box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
      border-radius: 8px;
    }
    h1 {
      text-align: center;
      color: #333;
    }
  </style>

```

```
label {
  font-weight: bold;
}
input, button {
  width: 100%;
  padding: 10px;
  margin-top: 10px;
  margin-bottom: 20px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box;
}
button {
  background-color: #007bff;
  color: white;
  border: none;
  cursor: pointer;
}
button:hover {
  background-color: #0056b3;
}
.results {
  margin-top: 20px;
}
.result-item {
  padding: 10px;
  border-bottom: 1px solid #ddd;
}
.result-item:last-child {
  border-bottom: none;
}
.result-title {
  font-weight: bold;
  color: #007bff;
}
.result-link {
  color: #555;
```

```

    word-break: break-all;
}
</style>
</head>
<body>
<div class="container">
  <h1>Reverse Video Search</h1>
  <form id="videoForm">
    <label for="videoUrl">Video URL:</label>
    <input type="url" id="videoUrl" placeholder="Enter video URL">

    <label for="videoUpload">Or Upload a Video:</label>
    <input type="file" id="videoUpload" accept="video/*">

    <button type="button" onclick="processVideo()">Process Video</button>
  </form>

  <div id="statusMessage"></div>

  <div class="results" id="resultsContainer">
    <h2>Sorted Results</h2>
  </div>
</div>

<script>
async function processVideo() {
  const videoUrl = document.getElementById('videoUrl').value;
  const videoUpload = document.getElementById('videoUpload').files[0];
  const statusMessage = document.getElementById('statusMessage');
  const resultsContainer = document.getElementById('resultsContainer');

  resultsContainer.innerHTML = '<h2>Sorted Results</h2>';
  statusMessage.textContent = "";

  if (!videoUrl && !videoUpload) {
    statusMessage.textContent = "Please provide a video URL or upload a video.";
    statusMessage.style.color = "red";
  }
}

```

```

    return;
}

statusMessage.textContent = "Processing video...";
statusMessage.style.color = "black";

try {
    let formData = new FormData();
    if (videoUrl) {
        formData.append("video_url", videoUrl);
    }
    if (videoUpload) {
        formData.append("video_file", videoUpload);
    }

    const response = await fetch("http://localhost:5000/process_video", {
        method: "POST",
        body: formData,
    });

    if (!response.ok) {
        throw new Error("Error processing video. Please try again.");
    }

    const data = await response.json();

    data.results.forEach((result, index) => {
        const resultItem = document.createElement("div");
        resultItem.classList.add("result-item");
        resultItem.innerHTML = `
            <div class="result-title">${index + 1}. ${result.title}</div>
            <a href="${result.link}" class="result-link" target="_blank">${result.link}</a>
            <p>Frequency: ${result.count}</p>
        `;
        resultsContainer.appendChild(resultItem);
    });
}

```

```
    statusMessage.textContent = "Processing complete! See results below.";
    statusMessage.style.color = "green";
  } catch (error) {
    statusMessage.textContent = error.message;
    statusMessage.style.color = "red";
  }
}
</script>
</body>
</html>
```